```
class Geeks {
    // Static variable
    static String geek = "Hi welcome!";

    Public static void main (String[] args)
    {
      // Access static without creating object
        sout (Geeks.geek);

      // static int c = 0;
      // Error: static variables cannot be declared
        inside a method.
    }
}
```

Java Operators :-

Arithmetic Operators : +, -, *, %, /

Logical Operators : &&, ||, !

Unary Operators : --, ++

Ternary Operators : cond ? true : false;

Assignment Operator : +=, -=, =, /=, %=, *=

Bitwise & Shift Operator : &, |, ^, <<, >>, >>>

Relational Operators : >, <, ==, <=, >=, !=

Instance of Operators : obj instanceof Integer

Bitwise Operators in java :-

*) perform operations directly on binary representation (bits) of integer values.

*) Bitwise operator work only with integer data types such as byte, short, int, long & char.

*) & -> Bitwise AND

| -> Bitwise OR

^ -> Bitwise XOR

~ -> Bitwise Complement (NOT)

<< -> Left shift

>> -> Signed Right Shift

>>> -> Unsigned Right Shift

1. Bitwise AND (&) :-

a = 5 => 0101 (Binary)

b = 7 => 0111 (Binary)

```
5 & 7 => 0101
       & 0111
         ----
         0101 => 5 (Decimal)
```

2. Bitwise OR (|) :-

a = 5 = 0101

b = 7 = 0111

a | b => 5 | 7 => 0101

| 0111

0111 => 7

3. Bitwise XOR (^) :-

a = 5 = 0101

b = 7 = 0111

a ^ b => 5 ^ 7 => 0101

^ 0111

1010 => 2

if different → 1

if same → 0

4. Bitwise Complement (~) :-

a = 5

32-bit binary respresentation of 5

00000000 00000000 00000000 00000101

0 becomes 1

1 becomes 0

In java, int is a 32-bit signed integer.

After applying ~a :-

11111111 11111111 11111111 11111010

= -6

This is because in java: ~N = -(N+1)

So: ~5 = -(5+1) = -6

Bit-Shift Operators :-

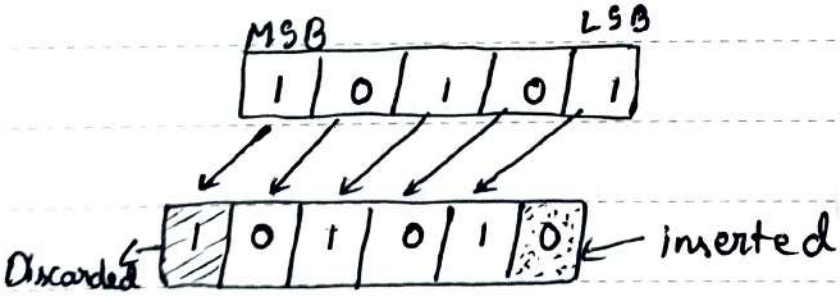
*) Shift the binary bits of a number left or right.

*) They work directly on binary data.

1. Signed Left Shift Operator (<<)
2. Signed Right Shift Operator (>>)
3. Unsigned Right Shift Operator (>>>)

Left Shift Operator in Java :- (<<)

Shifts each of the bits in binary representation toward the left. Multiplies power of 2.



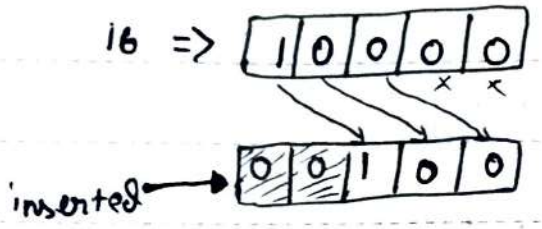
Binary	Decimal	
1 0 1	5	
1 0 1 0	10	→ left shift once
1 0 1 0 0	40	→ left shift twice

So, left shif x by n positions ⇒ $x * 2^n$

$\boxed{5 << 2}$ $5 * 2^1 = 10$, $5 * 2^2 = 40$

Signed Right Shift Operator :- (>>)

Shifts bits to the right. Which is effectively divides the number by ~~two~~ powers of two.



16 >> 2 => 4

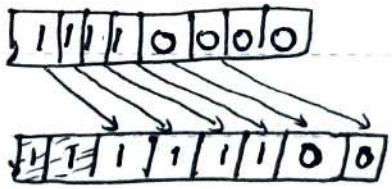
Right shift by 2 position.

*) The signed right shift operator (>>) shifts bits to the right while preserving sign bit (MSB)

- If number is positive, 0 is filled on left
- If number is negative, 1 is filled on left

a = -16

a>>2 =>



=> -4

Preserves the sign bit of a number and answer also in same sign.

Unsigned Right Shift Operator :- (>>>)

Shifts bits to right but always fills 0 on left, regardless of sign.

*) Does not preserve sign bit

*) Converts negative number into large positive number.

a = 16 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |

a >>> 2 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | => 4

Unsigned right shift by 2.

0 is inserted

a = -16 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 |

a >>> 2 | 0 | 0 | 1 | 1 | 1 | 1 | 0 | 0 |

32 16 8 4 2 1

write in 32-bit.

-16 => 11111111 11111111 11111111 11110000

after -16 >>> 2

00111111 11111111 11111111 11111100

=> 1073741820

Instanceof Operator :-

In java, instanceof is a keyword used for checking if a reference variable contains a given type of object reference or not.

It is also known as comparison operator where the Instance is getting compared to type returning boolean true or false.

```
class GFG {
    psvm()
    {
        GFG object = new GFG();
        sout(object instanceof GFG);
    }
}
```

O/P :- True.

```
class Parent {
    int value = 1000;
}

class Child extends Parent {
    int value = 10;
}

public class Main {
    psvm() {
  // 1. Child Object
    Child c = new Child();
    sout("1. Child instanceof Child:" + (c instanceof Child));
                                                    // True.
    sout("2. Child instanceof Parent:" + (c instanceof Parent));
                                                    // true.
    sout("3. Child instanceof Object:" + (c instanceof Object));
                                                    // true.
```

// Every class in java extends Objects

```
// 2. Parent Object
Parent p = new Parent();
Sout("4. Parent instanceof child" + (p instanceof child));
                                    // false.

// 3. Parent reference -> child object
Parent ref = new Child();
Sout("5. Parent ref instanceof child:" + (ref instanceof
                child)); // true.

// 4. null case
Parent nullObj = null;
Sout("6. null instanceof Parent:" + (nullObj instanceof
                Parent)); // false.

// 5. Safe typecasting using instanceof
if (ref Instanceof Child) {
    Child c2 = (child) ref;
    Sout("7. After typecasting, child value =" +
                c2.value); //10
```

```
//6. without typecasting (Shows parent variable)
    sout ("8. Parent reference value =" + ref.value); //1000
  }
}
```

Increment & Decrement Operators :-

Pre-inc	Post-inc	Pre-dec	Post-dec
inc ↓ replace	replace ↓ inc	dec ↓ replace	replace ↓ dec

Examples :-

1)

	x	y	value of x
y = ++x ⇒	10	11	11
y = x++ ⇒	10	10	11
y = --x ⇒	10	9	9
y = x-- ⇒	10	10	9

2) x = 5;
y = ++(++x); ×

error : Unexpected type

y => ++(++x) => ++(6)
⇓
This is not possible

What will be Output?

Error

"We can't 'inc' or 'dec' on constants"

y = ++(x+1); × error

++x => x = x + 1 ✓

++6 => 6 = 6 + 1 ×